

Assignment - 04

Title: Expression Tree

Objective: To study some advanced data structure such as Tree.

Problem Statement:

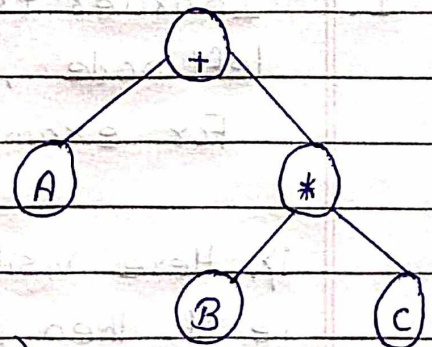
Construct an Expression tree from postfix and prefix expression, perform recursive and non-recursive inorder, pre-order and postorder traversals.

Outcome: Solve problem using algorithmic design techniques and data structure.

Theory:

Expression Tree $\Rightarrow$ An expression tree is a binary tree in which the operands are attached as leaf nodes and operators become the internal nodes.

For e.g.:-



From expression tree:

1. Inorder traversal: $A + B * C$ (LDR)
2. preorder traversal: $+ A * B C$ (D-L-R)
3. postorder traversal: $A B C * +$ (L-R-D)

- Inorder traversal of expression tree give's infix expression.
- The preorder traversal of expression tree give's prefix expression.
- The postorder traversal of expression tree give's postfix expression.

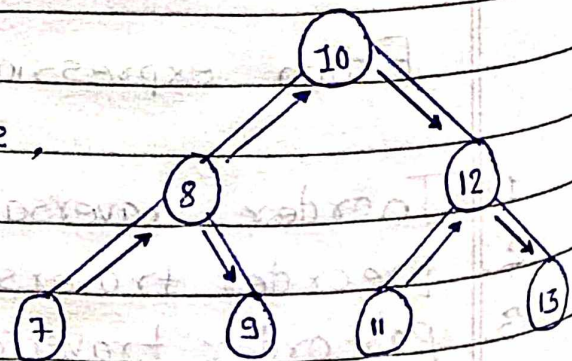
node representation of Expression tree :-

```
typedef struct node
{
    char data ;
    struct node * left ;
    struct node * right ;
}
```

There are Three traversal technique used for binary tree. These are inorder, preorder & postorder traversal.

1. Inorder traversal :- In this technique's the left node, parent and then right node is followed.
For example :

1). Here visit the leftmost node, i.e 7, then it's parent node 8, then 9 the right child will be visited.



- ii> The root node will be visited i.e visit 10.
- iii> Visit 11 then 12 and finally visit 13. Thus the traversal of right Subbranch is performed.

Hence, Inorder Sequence is 7, 8, 9, 10, 11, 12, 13.
In simple word's we can say that,
Inorder work's on LDR (left, Data, Right)

* Recursive inorder traversal :

Algorithm :

1. IF tree is not empty then
 - a. traverse the left Subtree in inorder.
 - b. visit the root node.
 - c. traverse the right Subtree in inorder.

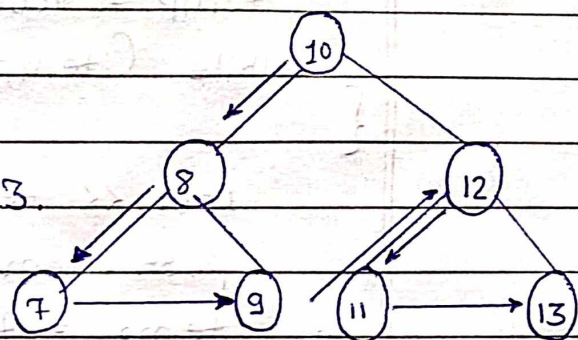
```
void inorder (node* (* temp))  
{  
    if (temp != NULL)  
    {  
        inorder (temp -> left);  
        cout << temp -> data;  
        inorder (temp -> right);  
    }  
}
```

* Non-recursive Inorder Traversal:-

```
int top = -1; // mean's Stack empty
if (root == NULL)
{
    cout << "Tree is empty";
    return;
}
temp = root;
while (temp != NULL)
{
    push(temp -> Top);
    temp = temp -> left;
}
if (!isEmpty(top))
{
    pop(top, temp);
    cout << temp -> data;
    temp = temp -> right;
}
```


2. preorder traversal : In this Method visit parent node the left and then right node.

- i>. Visit 10
- ii>. Visit 8, then 7 and finally 9
- iii>. Visit 12 then 11 and finally 13.



Hence, preorder traversal is
10, 8, 7, 9, 12, 11, 13.

* Recursive preorder traversal

Algorithm:

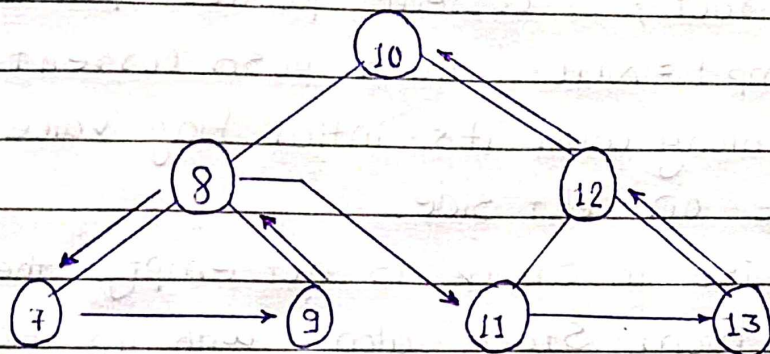
1. If tree is not empty then
 - a. visit the root node
 - b. traverse the left subtree in preorder
 - c. traverse the right subtree in preorder

```
void preorder (node * temp)
{
    if temp != NULL
    {
        cout << temp -> data ;
        preorder (temp -> left);
        preorder (temp -> right);
    }
}
```

Preorder non-recursive traversal

```
temp = root;
While (temp != NULL) {
    cout << temp->data;
    if (temp->rc != NULL)
    {
        push(temp->rc);
    }
    else
    {
        temp = temp->lc;
    }
}
```


3. post order Traversal: In this method, visit left node then right node and finally parent node.



The preorder sequence is:

7, 9, 8, 11, 13, 12, 10

* Recursive postorder traversal :-

Algorithm:

1. If tree is not empty then
 - a. traverse the left subtree in post order.
 - b. traverse the right subtree in post order.
 - c. visit the root node.

```

> { void postorder (node* temp)
    {
        if (temp != Null)
        {
            postorder (temp -> left);
            postorder (temp -> right);
            cout << temp -> data;
        }
    }
  
```


post-order non-recursive traversal :-

1. $temp = root$; // Current pointer pointing to root.
2. if $temp \neq NULL$ then push Current element pointer along with it's initial flag value on to Stack & traverse on left side.
3. Otherwise if Stack is not empty then pop an address from Stack along with it's flag value.
4. For the current pointer if the flag value is 'L' then change's it to 'R' & push Current pointer along with it's flag value 'R' on the Stack & traverse on right side.
5. Otherwise , for the element (current pointer) if the flag value is 'R' then display the element & make the current pointer NULL.
(i.e $temp = NULL$)
6. Repeat Step's 2, 3, 4, 5 until the stack becomes empty.


```
#include <iostream>

using namespace std;

struct Node
{
    char data;
    Node *lc, *rc;
};

class Tree
{
public:
    Node *root;

    Tree()
    {
        root = NULL;
    }

    void inorder_recursive(Node *);
    void preorder_recursive(Node *);
```

```
void postorder_recursive(Node *);  
void inorder_nonrecursive(Node *);  
void preorder_nonrecursive(Node *);  
void postorder_nonrecursive(Node *);  
Node *expression();
```

```
};
```

```
Node *Tree::expression()
```

```
{
```

```
Node *temp;
```

```
int i, top = -1;
```

```
char exp[20];
```

```
Node *stack[10];
```

```
int flag[10];
```

```
cout << "Enter the expression:";
```

```
cin >> exp;
```

```
for (i = 0; exp[i] != '\0'; i++)
```

```
{
```

```
if (exp[i] >= 'a' && exp[i] <= 'z')
```

```
{
```

```
temp = new Node;
```

```
temp->data = exp[i];
```

```
temp->lc = temp->rc = NULL;
```



```
stack[++top] = temp;
}
else
{
root = new Node;
root->data = exp[i];
root->lc = root->rc = NULL;
root->rc = stack[top--];
root->lc = stack[top--];
stack[++top] = root;
}
}
root = stack[top--];
return root;
}

void Tree::inorder_recursive(Node *root)
{
if (root != NULL)
{
inorder_recursive(root->lc);
cout << root->data;
inorder_recursive(root->rc);
```

```

}
}
void Tree::preorder_recursive(Node *root)
{
    if (root != NULL)
    {
        cout << root->data;
        preorder_recursive(root->lc);
        preorder_recursive(root->rc);
    }
}

void Tree::postorder_recursive(Node *root)
{
    if (root != NULL)
    {
        postorder_recursive(root->lc);
        postorder_recursive(root->rc);
        cout << root->data;
    }
}

void Tree::inorder_nonrecursive(Node *root)
{

```



```

struct Node *stack[20];
int top = -1;
while (root != NULL || top != -1)
{
    if (root != NULL)
    {
        stack[++top] = root;
        root = root->lc;
    }
    else
    {
        root = stack[top--];
        cout << root->data;
        root = root->rc;
    }
}

void Tree::preorder_nonrecursive(Node *root)
{
    struct Node *stack[20];
    int top = -1;
    stack[++top] = root;

```

```
while (top != -1)
{
    root = stack[top--];
    if (root != NULL)
    {
        cout << root->data;
        stack[++top] = root->rc;
        stack[++top] = root->lc;
    }
}

void Tree::postorder_nonrecursive(Node *root)
{
    struct Node *stack[20];
    int top = -1;
    int flag[10];
    while (top != -1 || root != NULL)
    {
        if (root != NULL)
        {
            stack[++top] = root;
            flag[top] = 0;
```



```
root = root->lc;
}
else
{
root = stack[top];
if (flag[top] == 1)
{
cout << root->data;
top--;
root = NULL;
}
else
{
flag[top] = 1;
root = root->rc;
}
}
}
}
int main()
{
Tree s;
```

```
s.root = s.expression();  
int ch, ans;  
cout << "\n*****MENU*****";  
cout << "\n1.Recursive function for inorder";  
cout << "\n2.Recursive function for preorder";  
cout << "\n3.Recursive function for postorder";  
cout << "\n4.Nonrecursive function for inorder";  
cout << "\n5.Nonrecursive function for preorder";  
cout << "\n6.Nonrecursive function forpostorder";  
do  
{  
cout << "\n ENTER YOUR CHOICE:";  
cin >> ch;  
switch (ch)  
{  
case 1:  
cout << "\nInorder Exp with Recursive =>";  
s.inorder_recursive(s.root);  
break;  
case 2:  
cout << "\npreorder Exp with Recursive =>";  
s.preorder_recursive(s.root);
```

```
break;
case 3:
cout << "\npostorder Exp with Recursive =>";
s.postorder_recursive(s.root);
break;
case 4:
cout << "\nInorder Exp with Non_Recursive =>";
s.inorder_nonrecursive(s.root);
break;
case 5:
cout << "\npreorder Exp with Non_Recursive =>";
s.preorder_nonrecursive(s.root);
break;
case 6:
cout << "\npostorder Exp with Non_Recursive =>";
s.postorder_nonrecursive(s.root);
break;
default:
cout<<"Invalid input";
break;
}
```



```
cout << "\nDo you want to continue (if yes then enter 1)" <<  
endl;  
cin >> ans;  
} while (ans == 1);  
cout<<"Exited !";  
return 0;  
}
```